

EXPEDITORS • ENGINEERING KICKOFF DAY

Git & GitHub Fundamentals

Everything you need on day one: the mental model, the daily workflow,
and your first push to GitHub.



\$ git init your-career

Initialized empty repository.

\$ git commit -m "Day 1: learn Git & GitHub"

[main a1b2c3d] Day 1: learn Git & GitHub

6 hours • live on Zoom

teach → demo → hands-on labs

GitHub Codespaces or local terminal

Agenda — 8:30 AM to 2:30 PM

MORNING

8 :30 Welcome + environment check

8 :45 M1 • The Git Mental Model

9 :25 **LAB 1 • Your First Repository**

9 :45 Break (10 min)

9 :55 M2 • Everyday Git Workflow

10 :30 **LAB 2 • Edit → Stage → Commit**

10 :55 M3 • GitHub Fundamentals

AFTERNOON

11 :30 Lunch (60 min)

12 :30 **LAB 3 • Local Meets Remote**

12 :55 M4 • Branching Basics

1 :30 **LAB 4 • Branch, Change, Merge**

1 :55 Break (10 min)

2 :05 M5 • Collaboration Preview

2 :22 Wrap-up • cheat sheet • Q&A

How today works

1 I explain

Short concept blocks with diagrams — the why before the how.

2 I demo

Live on my terminal / Codespaces. Watch the commands and their output.

3 You do

A hands-on lab after every module. Follow-along steps + a solo challenge.

WHAT YOU NEED TODAY

A GitHub account (free) — that's genuinely it. Labs run in GitHub Codespaces, a full dev machine in your browser. Prefer local? A terminal with Git 2.23+ works too.

MODULE 1 OF 5

The Git Mental Model

Goal — *understand what Git does before touching commands.*

`git status`

`git add`

`git commit`

`git log`



Why version control exists

LIFE WITHOUT IT

report.docx
report_v2.docx
report_final.docx
report_final_FINAL.docx
report_final_REAL_use-this-one.docx
copy of report_final(3).docx

Which one works? Who changed what? Can we go back?

LIFE WITH GIT



```
$ git log --oneline
8d3f2e1 Fix totals in Q4 summary
5b9c044 Add executive summary
21aa7f0 Rewrite intro after review
9e0d512 First draft of report
```

One file. Full history. Every version recoverable. Every change explained — and it works for a whole team at once.

What is Git?

Git is a **version control system**: it takes **snapshots** of your whole project every time you commit — like a save point in a video game you can always return to.

~94%

of professional developers use Git —
the de-facto industry standard

2005

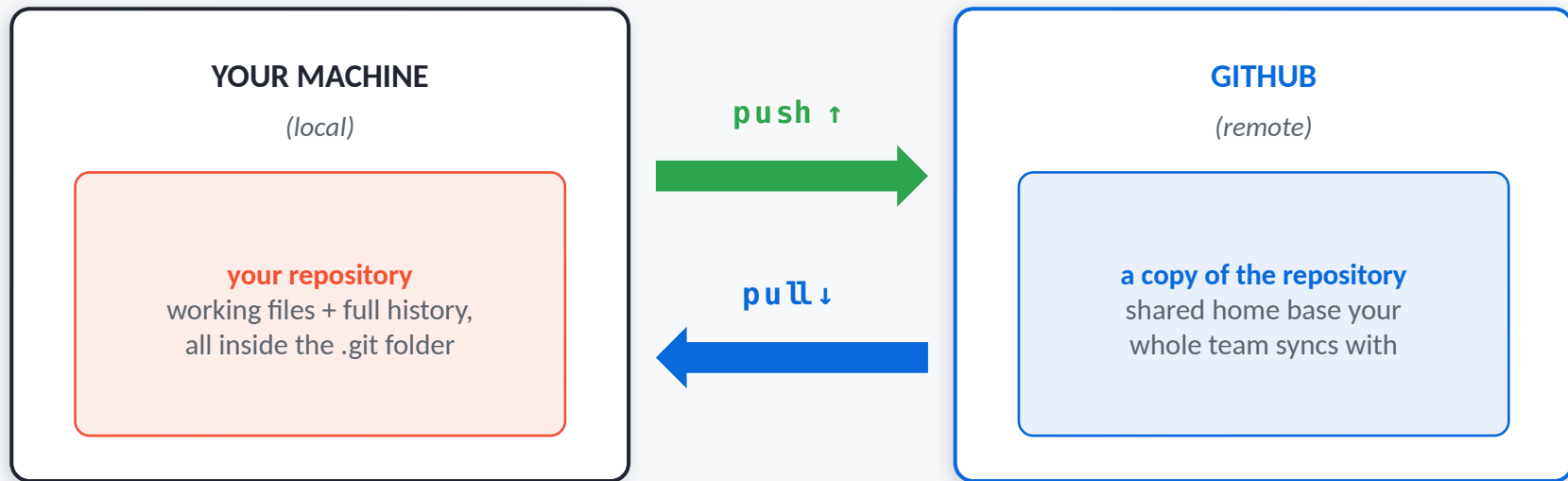
created by Linus Torvalds to manage
the Linux kernel

100%

local & free — full history on your
machine, works offline

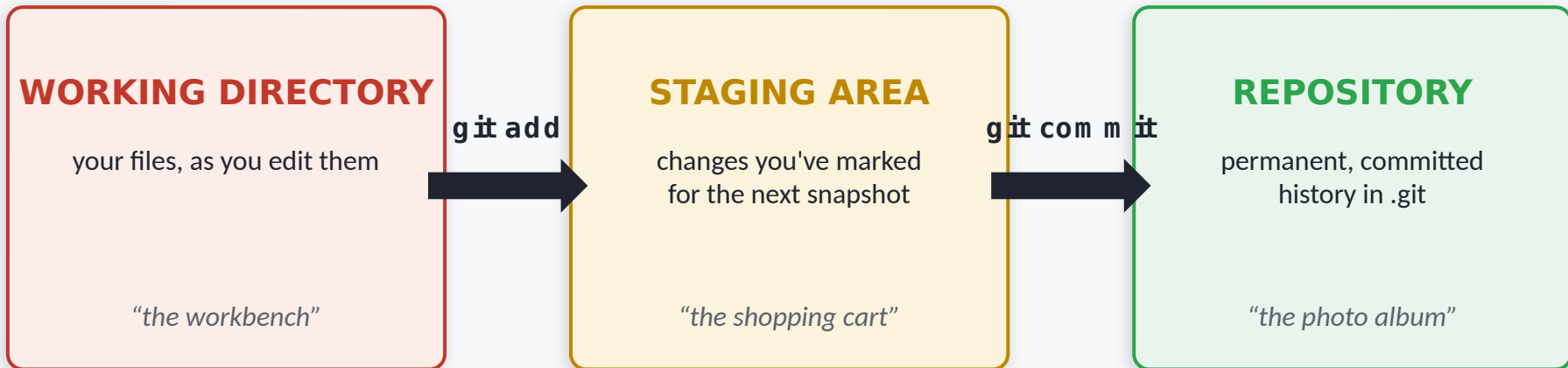
Git ≠ GitHub. Git is the tool on your machine. GitHub is a website that hosts Git repositories. More on that after lunch — for now, everything is local.

Local vs. remote



Nothing syncs automatically. Your laptop and GitHub are two separate repositories — they only exchange commits when you explicitly push or pull. This morning: 100% local. This afternoon: we connect the two.

The three areas



Changes flow one way: **edit** → **git add** → **git commit**. Why a staging area at all? So you choose exactly what goes into each snapshot — you can edit five files but commit two clean, focused changes.

Three commands you'll run constantly

`git status`

Where am I?

Shows what's changed, what's staged, and what Git suggests next. Free to run, impossible to break anything.

Run it before and after every other command — it's your GPS.

`git add`

Stage it

Moves a change from the working directory into the staging area. Nothing is saved yet — you're loading the cart.

`git add <file>` for precision • `git add .` for everything

`git commit`

Snapshot it

Permanently records everything staged, with a message and a unique ID (hash). This is the save point.

`git commit -m "Clear, imperative message"`

Anatomy of a commit

```
● ● ●  
$ git log  
commit 8d3f2e1a94c7b02f6e5d3a18c2b4f9e7...  
Author: Ada Lovelace <ada@expeditors.com>  
Date: Thu Jun 11 09:14:02 2026
```

Add shipment tracking endpoint

1

The hash — a unique fingerprint of this exact snapshot. First 7 characters are enough to refer to it.

2

Who & when — every commit is signed — that's why we ran git config on day one.

3

The message — the only part a human writes. Future-you and teammates depend on it.

A commit is a snapshot, not a diff. Git stores the state of every file at that moment — which is why jumping to any point in history is instant and safe.

How to read git status



```
$ git status
```

```
On branch main
```

```
Changes to be committed:
```

```
  new file:   invoice.py
```

```
Changes not staged for commit:
```

```
  modified:   README.md
```

```
Untracked files:
```

```
  notes.txt
```



GREEN staged — in the cart, will be in the next commit



RED (modified) tracked file changed, but not staged yet



RED (untracked) brand-new file Git isn't managing at all

Habit of the day: when in doubt, git status. It tells you the state and suggests the next command.

Module 1: watch first, then it's your turn

DURING THE DEMO, WATCH FOR...

- git status before git init — why the error?
- The hidden .git folder — where history lives
- Red → green → clean as one change travels through
- What git log shows after the first commit

HANDS-ON • 20 MIN

Lab 1 — Your First Repository

```
labs/lab-01-your-first-repository.m d
```

You're done when:

- ✓ A repo with 3 commits, made by you
- ✓ You can explain red vs. green in git status
- ✓ Bonus: you found your root commit's full hash

MODULE 2 OF 5

Everyday Git Workflow

Goal — build comfort with the edit → stage → commit cycle you'll run for the rest of your career.

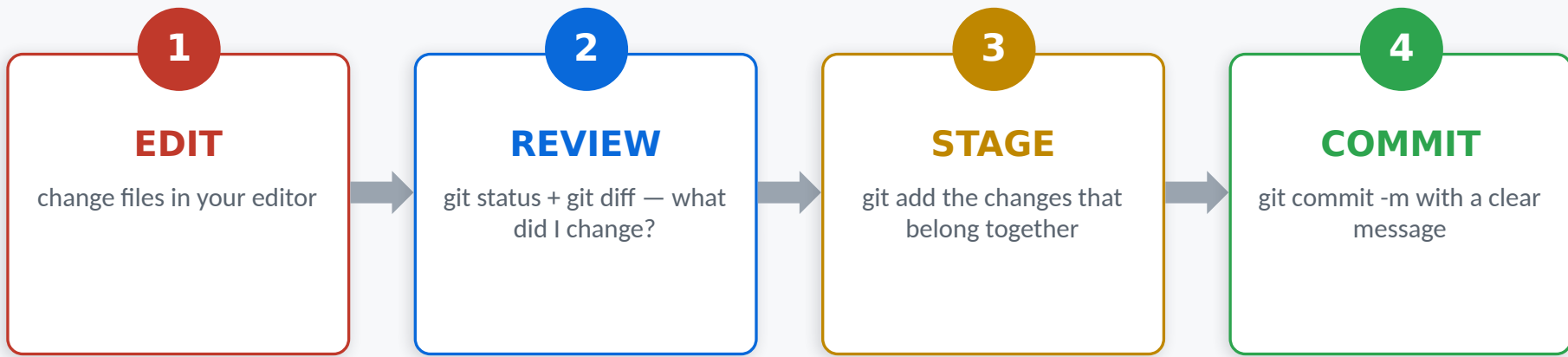
`git diff`

`git log --oneline`

`.gitignore`



The loop you'll run thousands of times



...and back to editing — small loops, many times a day

Commit small, commit often. Professionals commit several times an hour. Small commits = easy code review, easy bug hunting, easy undo.

git diff answers two different questions

git diff

“What have I edited but not yet staged?”

Compares working directory ↔ staging area. The moment you git add, the change leaves this view.

Run while editing

git diff --staged

“What exactly am I about to commit?”

Compares staging area ↔ last commit. Your final self-review before the snapshot becomes permanent.

Run right before every commit

Reading a diff: - removed line + added line — a changed line shows as one of each. A change lives in exactly one of the two views, never both.

The git log toolbox

git log

full detail: hash, author, date, message

reading one commit closely

git log --one-line

one commit per line

your daily driver — quick orientation

git log --one-line --graph

draws branch & merge structure

seeing how work flowed together

git log -p -2

full diffs for the last 2 commits

what exactly changed, line by line

Stuck in the pager? Long output opens in a scroll view — press **q** to quit. (Everyone hits this. Now you know.)

Commit messages that age well

X UNHELPFUL

"stuff"

"asdf"

"fixed it"

"changes"

"final commit (for real)"

✓ USEFUL

"Add custom form validation"

"Fix rounding error in totals"

"Remove unused login helper"

"Update README setup steps"

Imperative mood

"Add...", "Fix..." — matches Git's own style

Say what & why

a teammate should get it without reading code

~50 chars first line

short enough to scan in git log --oneline

.gitignore — what Git should never track

A plain text file of patterns. Anything matching is invisible to Git — it stays on your disk but never gets staged or committed. The **.gitignore itself IS committed**, so the whole team ignores the same things.

```
# .gitignore

*.log
# any log file, anywhere
.env
# secrets & credentials
node_modules/
# huge installed dependencies
build/
# generated output
```

Secrets — API keys, passwords, .env files — leaked keys on public GitHub get found by bots in seconds

Generated files — build output, logs, caches — recreated anyway, just noise in history

Dependencies — node_modules/ and friends — hundreds of MB that install from a list

⚠ **Gotcha:** .gitignore only affects untracked files — a pushed secret must be rotated, not just deleted.

Module 2: watch first, then it's your turn

DURING THE DEMO, WATCH FOR...

- A change visible in git diff... then 'vanishing' after add
- git diff --staged as the pre-commit review
- A fake API key — and how .gitignore hides it
- git log --oneline vs git log -p

HANDS-ON • 25 MIN

Lab 2 — Edit → Stage → Commit

```
labs/lab-02-everyday-workflow.md
```

You're done when:

- ✓ You used both diff views and can explain them
- ✓ A .gitignore hiding logs, secrets & build output
- ✓ Challenge: farewell() committed with a clean message

MODULE 3 OF 5

GitHub Fundamentals

Goal — *push your code to GitHub, pull it back down, and clone like it's day one on the job.*

`git push`

`git pull`

`git clone`

`git remote`



Git vs. GitHub

GIT

- the version control tool
- runs on your machine
- works fully offline
- made by Linus Torvalds, 2005
- free, open source

GITHUB

- a website that hosts Git repos
- runs in the cloud
- adds collaboration: PRs, reviews, issues
- owned by Microsoft, ~150M+ users
- one of several hosts (GitLab, Bitbucket)

Analogy: Git is to GitHub what photos are to Instagram — the thing itself vs. the place you share it.

What GitHub gives you

Backup & hosting

your repo, safe in the cloud, browsable from anywhere

Collaboration

many people, one codebase, without emailing zip files

Pull requests

propose → review → merge: how teams ship (preview at 2:05)

Issues & projects

track bugs, ideas and work alongside the code

Actions (CI/CD)

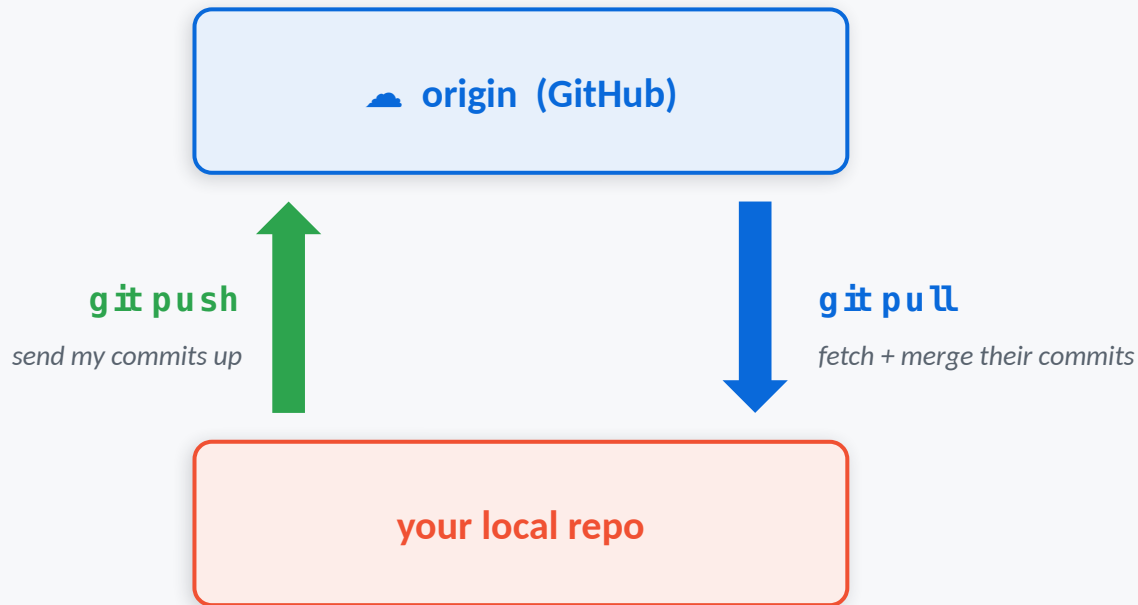
robots that test and deploy your code on every push

Codespaces

a full dev machine in your browser — you're using it today

Today we use: hosting + Codespaces. The rest is your roadmap.

Four verbs that move commits



git clone — first copy: entire repo + history, remote pre-wired. Day one at any job.

git fetch — download news without merging. Look before you leap; pull = fetch + merge.

Publishing an existing local repo

1

Create an EMPTY repo on GitHub

github.com → New repository. No README, no .gitignore — your local repo already has history; two histories collide.

in the browser — leave every checkbox unchecked

2

Tell Git where the remote is

“origin” is just the conventional nickname for the remote's URL.

```
$ git remote add origin https://github.com /you/repo.git
```

3

Push and link the branch

-u links local main to origin's main — after this, plain git push / git pull just work.

```
$ git push -u origin main
```

Then verify in the browser: refresh the repo page — your files, commits and hashes are there.

Authentication in 2026 (no passwords!)

GitHub removed password pushes in 2021. Today it's tokens managed for you — usually via a one-time browser sign-in.

TODAY

Codespaces

Already signed in. Pushing to your own repos just works. Zero setup — today's happy path.

LOCAL DEFAULT

HTTPS + credential manager

First push opens a browser sign-in; Git Credential Manager stores a token. Standard on Windows & macOS.

LATER

SSH keys / gh CLI

Key pair on your machine, or `gh auth login`. Preferred by many pros — a topic for later, not today.

If auth fights you in the lab, raise a hand fast — *it's the #1 first-day friction in real jobs too.*

init + connect vs. clone

PROJECT BORN ON YOUR MACHINE

git init

work, add, commit...

git remote add origin <url>

git push -u origin main

You had it first; GitHub gets a copy.

PROJECT ALREADY EXISTS SOMEWHERE

git clone <url>

...that's it. One command:

- ✓ full history downloaded
- ✓ origin remote pre-configured
- ✓ main checked out, tracking set

Day one at every job starts here.

Golden habit: pull before you start working, pull before you push. Most sync pain disappears.

Module 3: watch first, then it's your turn

DURING THE DEMO, WATCH FOR...

- Creating the GitHub repo EMPTY — and why
- `remote add + push -u`, then refresh the browser
- Editing on GitHub.com → my laptop has no idea → pull
- clone bringing down history + remote in one step

HANDS-ON • 25 MIN

Lab 3 — Local Meets Remote

```
labs/lab-03-local-meets-remote.md
```

You're done when:

- ✓ Your repo is live on github.com
- ✓ You round-tripped a change: web edit → pull
- ✓ Challenge: synced two local copies through GitHub

MODULE 4 OF 5

Branching Basics

Goal — *understand branches well enough to stop fearing them.*

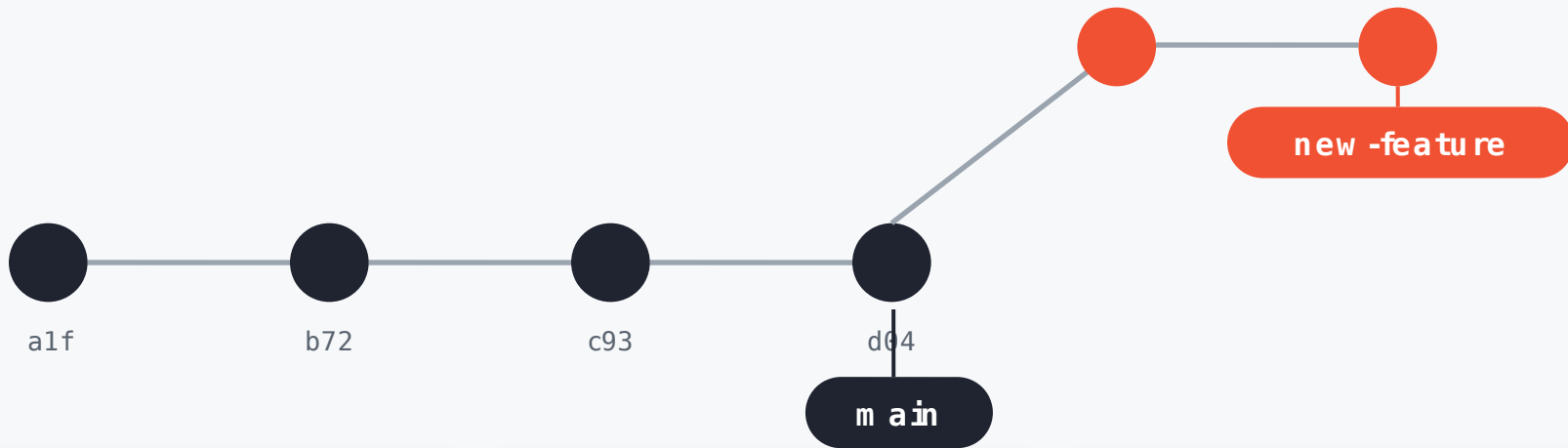
`git switch -c`

`git merge`

`git branch -d`



A branch is just a movable pointer



41 bytes

a branch is literally a tiny file holding one commit hash — not a copy of your code

instant

creating or switching costs nothing; Git just repaints your folder from its database

fearless

commits on a branch are as safe as commits anywhere — branches protect work, not endanger it

Why branches exist

Experiment safely — try anything on a branch — main is untouched until you decide to merge

Work in parallel — five teammates, five branches, zero stepping on each other

Keep main shippable — the team's rule: main always works. All work happens on branches

Real-team rule you'll meet everywhere: nobody commits directly to main. Every change rides in on a branch.

```
$ git branch
* main

$ git switch -c fix-totals
Switched to a new branch
'fix-totals'

$ git branch
* fix-totals
  main
```

Creating and switching branches

git switch -c <name>

create a new branch and move onto it (-c = create)

git switch <name>

move to an existing branch

git branch

list branches; * marks where you are

git branch -d <name>

delete a merged branch (it refuses if unmerged — safe)

THE “WHOA” MOMENT

Switch branches and files appear or vanish from your folder. Nothing is deleted — Git repaints your working directory to match the branch's snapshot. Your work lives in commits, not in the folder.

Seen git checkout in old tutorials? Same job. switch (2019) is the modern, single-purpose command — use it; translate checkout when you read it.

Merging: two shapes of history

FAST-FORWARD

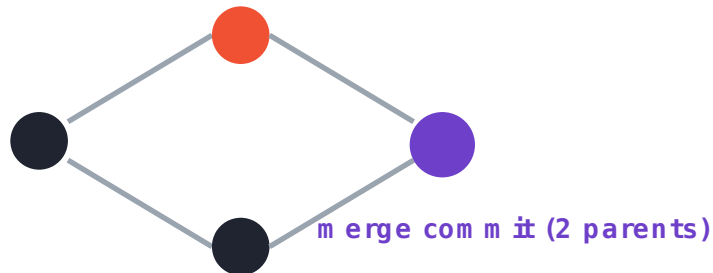
main didn't move while you worked → Git slides the pointer forward. No new commit.



`$ git merge` → “Fast-forward”

MERGE COMMIT

both branches moved → Git creates one new commit with two parents, tying history together.

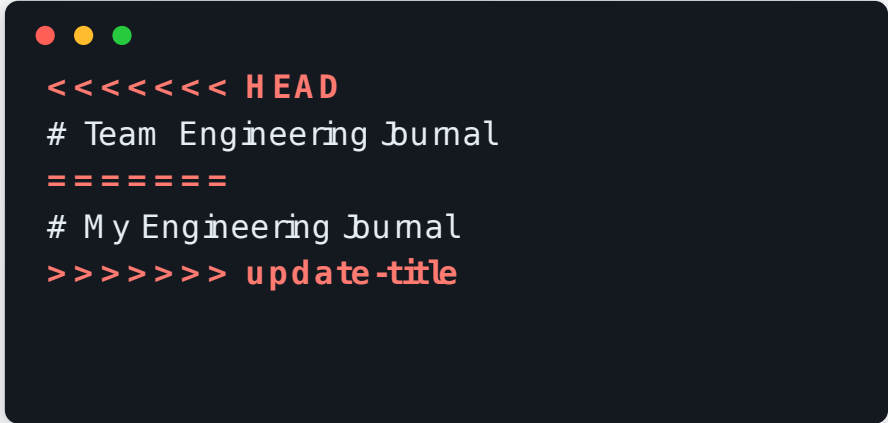


`$ git merge` → “Merge made by ort”

One rule to remember: switch to the branch that should RECEIVE the changes, then `git merge <branch>`

Merge conflicts: a question, not an error

When two branches change the **same lines**, Git won't guess. It pauses the merge and asks you to choose. Different files or different lines? Git merges silently — conflicts are rarer than their reputation.



```

<<<<<< HEAD
# Team Engineering Juma1
=====
# My Engineering Juma1
>>>>>> update-title

```

between <<<<<< and ===== :

your current branch's version

between ===== and >>>>>> :

the incoming branch's version

1. open the file

2. keep what's right

3. delete the markers

4. add + commit

Module 4: watch first, then it's your turn

DURING THE DEMO, WATCH FOR...

- A file disappearing as I switch branches (and coming back)
- Fast-forward merge — watch the pointer slide
- A conflict made on purpose — and resolved calmly
- Deleting a branch ≠ deleting work

HANDS-ON • 25 MIN

Lab 4 — Branch, Change, Merge

```
labs/lab-04-branch-change-merge.md
```

You're done when:

- ✓ You branched, committed, merged, and cleaned up
- ✓ You resolved one guided conflict — and survived
- ✓ Challenge: you can explain FF vs. merge commit

MODULE 5 OF 5

Collaboration Preview

Goal — *see where this is heading — how real teams turn branches into shipped software.*

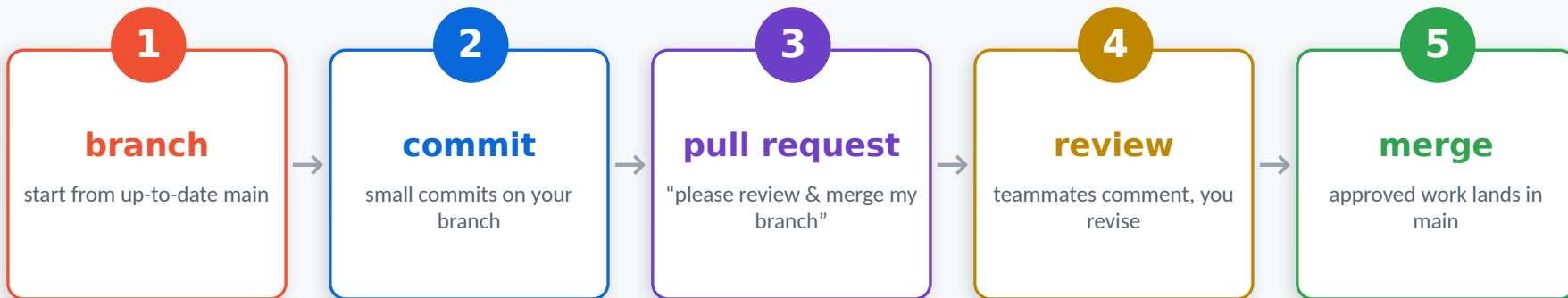
pull requests

code review

protected main



How real teams ship code



...then everyone pulls the new main, branches again, and the loop continues.

Today you already did steps 1, 2 and 5 — solo. A pull request is just the team version of the merge you ran in Lab 4: same merge, plus a conversation in the middle. That conversation is the next session.

What a pull request actually is

A pull request (PR) is a **request to merge one branch into another** — wrapped in a discussion page on GitHub. It's not a Git command; it's a GitHub feature built on top of branches and merges you already know.

A diff anyone can read — every changed line, side by side, in the browser

A conversation — comments on specific lines; questions before code lands

A quality gate — approvals and automated tests must pass before merge

A paper trail — months later: what changed, who, and —crucially—why

Why “pull” request? You're asking the project to pull your branch in. (Odd name, universal term.)

main is sacred — and usually locked

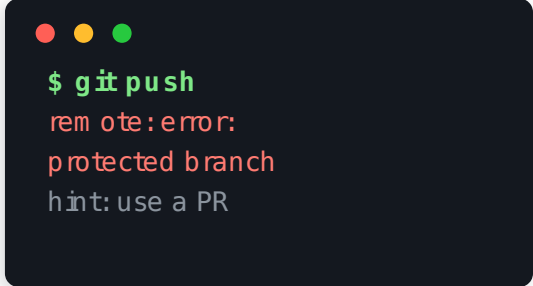
WHAT “MAIN” MEANS ON A TEAM

The shared, always-working version of the project — what ships to customers.

Branch protection (a GitHub setting) typically enforces:

- no direct commits to main — PRs only
- at least one approving review
- automated tests must pass first

SO WHEN YOU SEE...

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The text inside the terminal is as follows:

```
$ git push  
remote: error:  
protected branch  
hint: use a PR
```

...it's not broken. It's the team's safety system saying: branch, then PR.

TAKE THIS WITH YOU

Your day-one cheat sheet

git init

start a repo here

git clone <url>

copy a remote repo

git status

where am I? (run constantly)

git push / pull

send / receive commits

git add <f> / .

stage changes

git remote -v

where is my remote?

git commit -m "..."

snapshot what's staged

**git switch -c **

new branch + switch

git diff / --staged

unstaged / staged changes

**git merge **

merge into current

git log --oneline

compact history

**git branch -d **

delete merged branch

Stuck? git status first. Then ask. Nothing committed is ever truly lost.

YOU SHIPPED TODAY

From zero to push in one day

- ✓ a repo with real history, built commit by commit
- ✓ the three-areas model — you know where every change lives
- ✓ your code on GitHub, pushed, pulled and cloned
- ✓ a branch merged (and one conflict survived)

KEEP GOING

Pro Git (free book) — git-scm.com/book

GitHub Skills — skills.github.com

When it goes wrong — ohshitgit.com

Next session: pull requests & team workflow



```
$ git commit -m "Day 1 complete — Git fundamentals unlocked"
$ git push # see you at the next session!
```